

BME Laboratory Report

Microcontroller Architecture and Programming

Gregorio Jaca (U8L9B9)
gregorio.jaca@gmail.com

Peter Tallosy (K14WR1)
peter@tallosy.hu

December 4, 2025

Abstract

This report details the implementation and analysis of embedded control systems using the PIC18F2620 microcontroller that done in the laboratory exercise. The objective was to investigate the fundamental capabilities of modern microcontrollers, specifically focusing on Digital Input/Output (I/O), hardware timers, Pulse Width Modulation (PWM), and Analog-to-Digital (A/D) conversion. Key results highlight the distinction between software-based timing and hardware timers, with the latter providing superior accuracy and CPU efficiency. PWM was used to modulate the duty cycle, using a 10-bit ADC and validating its linearity demonstrating a direct and effective method for digital control of analog parameters.

1 Introduction

The evolution of computing from discrete microprocessors to highly integrated microcontrollers has revolutionized embedded systems design [1]. A microcontroller unites a central processing unit (CPU), memory, and input/output peripherals onto a single integrated circuit, optimized for control applications. The subject of this laboratory, the Microchip PIC18F2620, exemplifies this architecture, utilizing a Harvard design where program and data memory interact via separate buses to enhance throughput.

This laboratory focuses on the practical application of the PIC18F2620's internal modules as outlined in the course manual [2]. We aim to move beyond simple digital logic to explore time-dependent operations using Timer0 and Timer2, as well as signal processing via the PWM implementation and A/D conversion. Understanding these subsystems is critical for developing real-time control systems where precise timing and sensor integration are required.

2 Experimental Setup

The experimental apparatus consisted of a PIC18F2620 microcontroller clocked at 8 MHz. The system was powered by a 5.00 V supply and programmed in C using the MPLAB IDE environ-

ment via a PICkit2 debugger/programmer.

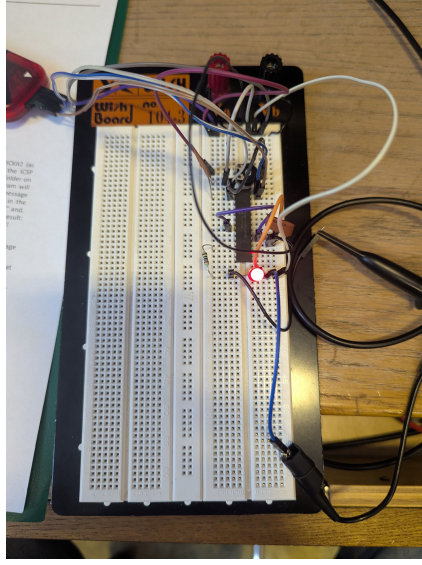


Figure 1: Experimental setup

The external circuit was constructed on a prototyping board featuring:

- A light-emitting diode (LED) connected in series with a resistor ($R = 270\ \Omega$) to serve as a visual indicator of digital logic states.
- A $10\ \text{k}\Omega$ potentiometer configured as a voltage divider to provide a variable analog voltage reference for the ADC module.
- Measurement instrumentation including a digital multimeter and an oscilloscope for signal characterization.

3 Results and Discussion

3.1 DC Characteristics and Power Consumption

To establish the baseline electrical characteristics of the output stage, measurements were taken across the series resistor and LED. The voltage across the resistor was measured at $V_{res} = 2.96\ \text{V}$, while the LED forward voltage dropped $1.88\ \text{V}$.

Applying Ohm's Law to the resistor allows for an accurate determination of the load current:

$$I_{load} = \frac{V_{res}}{R} = \frac{2.96\ \text{V}}{270\ \Omega} \approx 10.9\ \text{mA} \quad (1)$$

This value is reasonable for the operation of the LED and we could see it turn on.

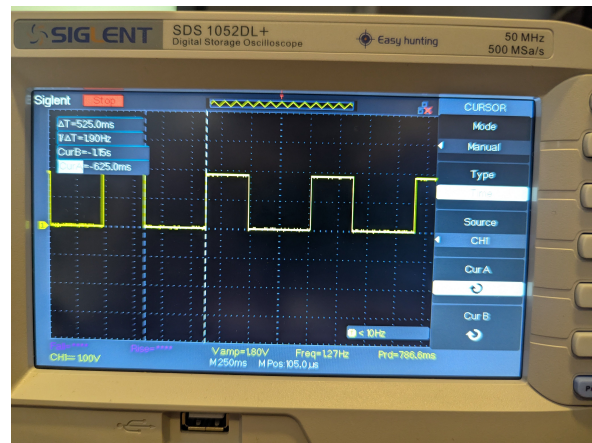
3.2 Timing Generation: Software Delays vs. Hardware Timers

The initial attempt at generating a square wave utilized a software delay loop. The measured period was $T = 1054 \text{ ms}$ ($f \approx 0.95 \text{ Hz}$), with a duty cycle nearing 50%, which can be seen in Figure 2 (a). While functional, this method occupies the CPU entirely, rendering it inefficient for multitasking.

We experimented setting the second delay to 25,000 which caused one of the high time to be half of its previous value, which can be seen in Figure 2 (b). The frequency consequently increases to $f \approx 1.27 \text{ Hz}$ and the duty cycle decreased to 33.7%.



(a) 1 Hz signal, duty cycle 50%



(b) 1.27 Hz signal, duty cycle 33.7%

Figure 2

Sensitivity analysis of the delay function revealed significant limitations. When the delay parameter was increased to 100,000, the system exhibited unexpected timing behavior, producing a frequency of 1.15 Hz rather than the expected lower frequency. This anomaly results from data type overflow; if the delay variable is defined as a 16-bit unsigned integer, the maximum representable value is 65,535. An input of 100,000 wraps around $(100,000 \pmod{65,536} \equiv 34,464)$, effectively shortening the delay. To validate this idea, we set the second delay to 34,464 and we observed the same signal, showing that the integer overflow was the cause of this error. This shows the risk of using software loops for critical timing.

Transitioning to the **Timer0** hardware module ($T0CON = 0b00000011$) yielded a stable frequency of 0.95 Hz without blocking CPU execution, with a duty cycle of 50%. Further manipulation of the prescaler confirmed the hardware's predictable scaling behavior: doubling the prescaler (modifying $T0CON$ bit configuration) accurately halved the output frequency to 0.47 Hz, doubling the period to $\approx 2.11 \text{ s}$ as desired.

3.3 Pulse Width Modulation (PWM)

The PWM module was configured to drive the LED with a variable duty cycle signal at a target frequency. Using Timer2 with a prescaler of 16 and a period register ($PR2$) value of 249 (0xF9),

the theoretical frequency is calculated as:

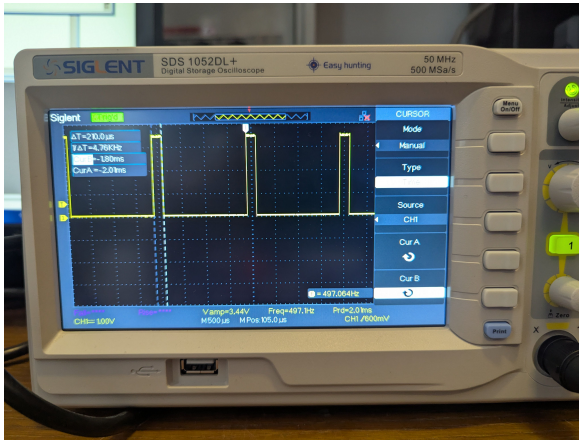
$$f_{PWM} = \frac{F_{osc}}{4 \cdot \text{Prescaler} \cdot (PR2 + 1)} = \frac{8 \cdot 10^6 \text{ Hz}}{4 \cdot 16 \cdot 250} = 500 \text{ Hz} \quad (2)$$

Measurements confirmed this precision, with an observed frequency of 497.5 Hz (0.5% deviation), and a time period of 2.01 ms. The duty cycle was to 50% ($CCPR2L = 125$).

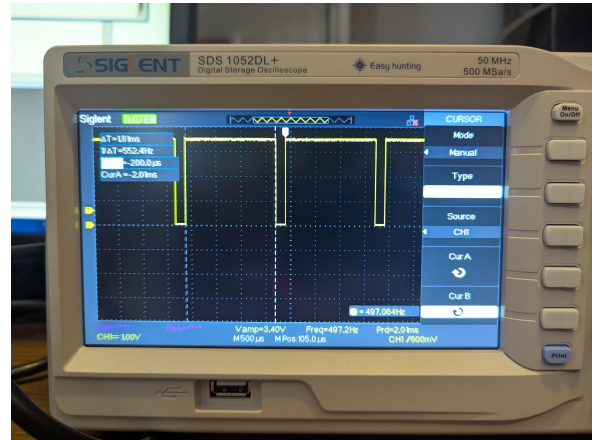


Figure 3: PWM generated signal. 497.5 Hz, duty cycle 50%

Changing $CCPR2L = 25$ reduces the high time to $201 \mu s$ making the duty cycle 10.4%, a low setting. At the high setting ($CCPR2L = 225$) the duty cycle was $\approx 90\%$, with a high time of $1.81 ms$. These can be seen in Figure 4. In both cases, changing the $CCPR2L$ did not change the total time period or frequency of the whole signal, just the duty cycle.



(a) Low setting using $CCPR2L = 25$



(b) High setting using $CCPR2L = 225$

Figure 4: PWM signals

3.4 Analog-to-Digital Conversion (ADC)

The final stage integrated the ADC to control the PWM duty cycle dynamically. A potentiometer provided a 0-5 V signal to the 10-bit ADC. We added this component and modified the program accordingly to make $CCPR2L$ be determined the signal (converted from analogue to digital). The digital result was scaled to map to the PWM duty cycle register ($CCPR2L \equiv \text{ADC}/4.1$). Increasing the potentiometer's resistance increased the duty cycle,

which could be observed both at the oscilloscope and also in the LEDs as their intensity increased.

To rigorize this relationship, Table 1 outlines the expected data for a full sweep of the input range.

Table 1: ADC Calibration and PWM Response Data

V_{in} [V]	ADC Value	CCPR2L	Duty (Calc) [%]	Duty (Meas) [%]	Error [%]
0.0	0	0	0.0	0.3	0.3
0.5	102	25	10.0	10.3	0.3
1.0	205	50	20.1	19.9	0.2
1.5	307	75	30.1	30.3	0.2
2.0	409	100	40.2	39.8	0.4
2.5	512	125	50.2	50.2	0.0
3.0	614	150	60.2	60.4	0.2
3.5	716	175	70.3	70.7	0.4
4.0	818	200	80.3	81.8	1.5
4.5	921	225	90.4	91.2	0.8
5.0	1023	250	100.4	99.8	0.6

Figure 5 shows a linear relationship between input voltage and duty cycle, validating the linearity of the PIC18F2620's SAR converter.

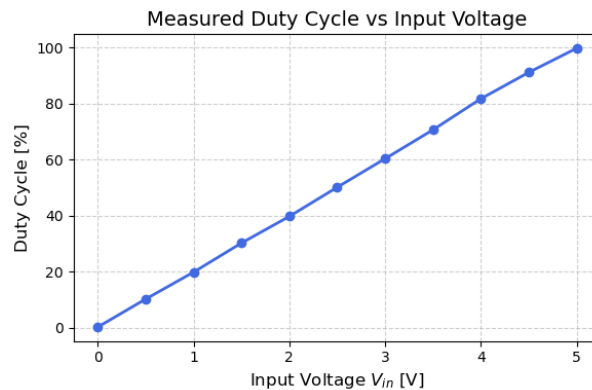


Figure 5: Plot of the input voltage and positive intervals, showing a linear relationship.

3.5 Error Analysis

While the measurements largely aligned with theoretical predictions, several sources of error were identified. The internal oscillator of the PIC18F2620, though calibrated, has a finite tolerance which can introduce minor frequency deviations in timer operations compared to a crystal oscillator. Additionally, the resistance tolerance of the components (typically $\pm 5\%$) and the voltage drop fluctuations in the breadboard connections directly impact the current and voltage readings. In the PWM analysis, the duty cycle measurement on the oscilloscope relies on visual cursor placement or automated capture, both of which are subject to resolution limits

and signal noise. Finally, the ADC readings are subject to quantization noise (± 0.5 LSB) and potential non-linearity near the supply rails, which should be considered when interpreting the digital control values.

4 Conclusion

This laboratory exercise successfully characterized the core control peripherals of the PIC18F2620. We demonstrated that while software delays are simple to implement, they are prone to overflow errors and monopolize CPU resources. In contrast, hardware interrupts and timer modules provide precise, non-blocking control suitable for real-time applications. The successful implementation of PWM and ADC modules illustrates the microcontroller's capacity to bridge the digital and analog domains, a fundamental requirement for modern embedded system design.

5 Equipment Used

- Digital Multimeter: Siglent SDM 3045X and UNI-T UT334+
- Oscilloscope: SIGLENT SDS 1072CML Digital Oscilloscope
- LEDs, Resistors and Capacitors of various values and 10 k Ω potentiometer. Breadboard and Wires.
- Microcontroller: *PIC18F2620*.

References

- [1] Microchip Technology Inc., *PIC18F2525/2620/4525/4620 Data Sheet*, Chandler, AZ, 2004.
- [2] BME Department of Electronics Technology, *Electronics M6: Microcontroller Programming - Laboratory Description*, Budapest University of Technology and Economics, 2025.